Mining Image Features for Efficient Query Processing

Beitao Li, Wei-Cheng Lai, Edward Chang and Kwang-Ting Cheng
Electrical & Computer Engineering
University of California, Santa Barbara
echang@ece.ucsb.edu

Abstract

The number of features required to depict an image can be very large. Using all features simultaneously to measure image similarity and to learn image query-concepts can suffer from the problem of *dimensionality curse*, which degrades both search accuracy and search speed. Regarding search accuracy, the presence of irrelevant features with respect to a query can contaminate similarity measurement, and hence decrease both the recall and precision of that query. To remedy this problem, we present a mining method that learns online users' query concepts and identifies important features quickly. Regarding search speed, the presence of a large number of features can slow down query-concept learning and indexing performance. We propose a divide-and-conquer method that divides the concept-learning task into G subtasks to achieve speedup. We notice that a task must be divided carefully, or search accuracy may suffer. We thus propose a genetic-based mining algorithm to discover good feature groupings. Through analysis and mining results, we observe that organizing image features in a multi-resolution manner, and minimizing intra-group feature correlation, can speed up query-concept learning substantially while maintaining high search accuracy.

Keywords: data mining, query concept, relevance feedback.

1 Introduction

We explore two multimedia data mining problems in this paper. The first problem is to mine a user's query concept. A user who searches for multimedia content often encounters difficulty articulating his/her query concept. We present an intelligent mining method that can grasp the target concept quickly with a small number of labeled instances. The second mining problem that we tackle is to discover the optimal grouping of multimedia features (attributes) to achieve learning speedup while maintaining high search accuracy.

Mining a user's query concept is critical for online multimedia search such as looking up an e-commerce product or finding a desired image. In these applications, a query concept is difficult to articulate, and articulation can be subjective. For instance, a user may find it difficult to

depict an image in low-level features, and different people may describe and interpret the same image differently. To remedy this query articulation difficulty, many search engines [10] require users to provide examples to seed a query. (For example, providing sunset images as examples in order to search for sunset pictures.) However, finding good examples is the job of a search engine itself, and this circular requirement leaves the core problem—understanding users' query concepts—unsolved.

We would like to use mining techniques that not only can learn users' query concepts, but also can grasp the concepts quickly. Unfortunately, traditional data mining algorithms and techniques are not suitable for this task for at least two reasons:

1. *Time and sample constraints.* Traditional learning methods such as decision trees and neural networks require a large number of training instances and can take a long time (more than a few seconds) to learn a concept in a high-dimensional feature space. But, online users are often too impatient to wait around or to provide a great deal of feedback.

2. *Limited room for preprocessing.* Traditional mining techniques such as user profiling and feature reduction may not be helpful for online query-concept learning. First, a user's query concept may be independent of his/her past queries. Profiling users, or using their past query patterns to predict future query concepts, can be counter-productive. In addition, performing feature reduction is infeasible in this online learning setting, since not all query concepts can be known in advance. (Performing feature reduction requires knowing all possible concepts, i.e., instance labels.)

In this paper, we first review a query-concept learner, MEGA, that we proposed in [2] to learn a concept through an intelligent sampling process. MEGA models query concepts in k-CNF [6], which can model virtually all practical query concepts[1]. Learning a k-CNF, however, can be time consuming. For instance, when the number of features is larger, a k-CNF concept with a moderate k can have millions of terms. The focus of this paper thus aims to reduce learning time of MEGA by dividing the learning task into G subtasks (to achieve a speedup of $O(G^{k-1})$ folds). At first sight, dividing features seems to be straightforward. For instance, many systems divide image features into color, texture, and shape sets. Our empirical study, however, shows that this "natural" division can often yield poor search precision. We employ a genetic-based algorithm to discover good feature groupings that yield good precision. Our study using a 144-dimension

[1] k-CNF is more expressive than k-term DNF, and it has both polynomial sample complexity and time complexity [7, 8].

diversified image dataset shows that the groupings that achieve high precision tend to have low intra-group feature correlation. As a part of our study, we also discovered k-CNF expressions for various image categories. We present and compare some selective learned concepts and discuss how concept-feature associations can be used for enhancing image search and similarity measurement.

The rest of this paper is organized into four sections. Section 2 defines notations and briefly reviews MEGA. Section 3 depicts the feature-division problem and presents the genetic-based algorithm for mining the optimal grouping. Section 4 presents our experimental results. Finally, we offer our conclusions in Section 5.

2 Query-Concept Learning

To provide the context, this section defines notations and reviews MEGA—a concept learning algorithm that we present in [2].

2.1 Definitions and Notations

Our learner models query concepts in k-CNF and uses k-DNF to bound the uncertain region for sampling.

Definition 1: k-**CNF**: For constant k, the representation class k-CNF consists of Boolean formulae of the form $c_1 \wedge \cdots \wedge c_\theta$, where each c_i is a disjunction of at most k literals over the Boolean variables $x_1, \ldots, x_n$. No prior bound is placed on θ.

Definition 2: k-**DNF**: For constant k, the representation class k-DNF consists of Boolean formulae of the form $d_1 \vee \cdots \vee d_\theta$, where each d_i is a conjunction of at most k literals over the Boolean variables $x_1, \ldots, x_n$. No prior bound is placed on θ.

In our retrieval system, queries are boolean expressions consisting of predicates connected by the boolean operators $\vee$ (or) and $\wedge$ (and). A predicate on feature x_k in our system is in the form of P_{x_k}. Note that the membership function [3] of P_{x_k} can be either crispy or fuzzy. A database system consists of a number of predicates. Our goal is to find the proper operators to combine individual predicates to form the query concept. We choose k-CNF to model query concepts, since it can express most practical queries and can be learned via positive-labeled instances in polynomial time [6, 8]. In addition, we use non-positive-labeled instances to reduce our sampling space (bounded by k-DNF), which we will discuss in detail in Section 2.2.

A k-CNF has three characteristics:

1: The terms (or literals) are combined by the $\wedge$ operator.

2: The predicates in a term are combined by the $\vee$ operator.

3: A term can have at most k predicates.

Suppose we have three predicates P_{x_1}, P_{x_2} and P_{x_3}. The 2-CNF of these predicates is

$$P_{x_1} \wedge P_{x_2} \wedge P_{x_3} \wedge (P_{x_1} \vee P_{x_2}) \wedge (P_{x_1} \vee P_{x_3}) \wedge (P_{x_2} \vee P_{x_3}).$$

Suppose a dataset contains N objects, denoted as O_i, where $i = 1 \ldots N$. Suppose each object can be depicted by M features, each of which is denoted by x_k, where $k = 1 \ldots M$. To find objects that best satisfy a k-CNF concept, we need to compute the membership value of an object with respect to the concept. We first compute the membership values at predicate level and then aggregate them into the membership value at concept level. At the predicate level, suppose $F_{x_k}(O_i)$ is the value of object i on feature x_k. We transform $F_{x_k}(O_i)$ to the predicate level membership value using a membership function μ: $F_{x_k}(O_i) \rightarrow [0, 1]$. Let $P_{x_k}(O_i)$ denote the predicate level membership value of object i for predicate P_{x_k}. When $P_{x_k}(O_i) = 1$, it means that object i fully satisfies predicate P_{x_k}. When $P_{x_k}(O_i) = 0$, it means that object i does not satisfy predicate P_{x_k} at all, and when $0 < P_{x_k}(O_i) < 1$, it means that object i satisfies predicate P_{x_k} to a degree. For example, people's heights can vary from 3 feet to 9 feet. If we want to grasp the fuzzy predicate of "tall", we use a membership function to transform each height value between 3 feet and 9 feet to a membership value between 0 and 1. If we consider people over 7 feet to be definitely tall, then we map heights over 7 feet to 1.0. If we consider people under 5 feet definitely not tall, then we map heights under 5 feet to 0.0. Heights between 5 and 7 feet are considered tall to a degree, and we map them to membership values between 0 and 1.

At the concept level, we use the standard fuzzy rules, as defined by Zadeh [4, 11], to aggregate membership values on individual predicates. To combine M predicate level membership values into one concept level membership value, we need an *M-ary* aggregation function that maps $[0, 1]^M$ to $[0, 1]$. The rules are as follows:

Conjunctive rule: $P_{x_1 \wedge x_2 \wedge \ldots \wedge x_M}(O_i) = \min\{P_{x_1}(O_i), P_{x_2}(O_i), \ldots P_{x_M}(O_i)\}$.

Disjunctive rule: $P_{x_1 \vee x_2 \vee \ldots \vee x_M}(O_i) = \max\{P_{x_1}(O_i), P_{x_2}(O_i), \ldots P_{x_M}(O_i)\}$.

The objects that best satisfy a concept have the largest aggregated membership values.

2.2 The MEGA Algorithm

MEGA models query concepts in k-CNF, which can model virtually all practical query concepts. MEGA consists of two major parts:

- Refinement: Refine the query concept based on positive-labeled instances. The refinement step is carefully designed to tolerate noisy data.
- Sampling: Refine the sampling space based on negative-labeled instances and select samples judiciously for expediting the learning process.

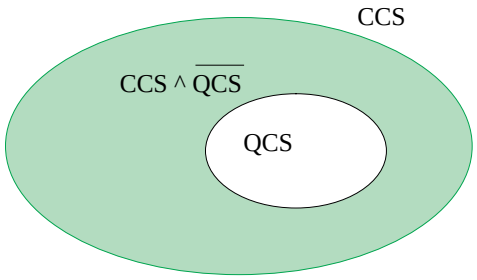


Figure 1: MEGA's Sampling Space: $CCS \wedge \overline{QCS}$.

MEGA initializes the query concept space (QCS) as a k-CNF and the candidate concept space (CCS) as a k-DNF. The QCS starts as the most specific concept and the CCS as the most general concept. The target concept that MEGA learns is more general than the initial QCS and more specific than the initial CCS. MEGA learns the QCS, while at the same time refining the CCS to delimit the boundary of the sampling space. (The shaded area in Figure 1 shows the sampling space between the QCS and the CCS). To ensure that target concepts can be learned quickly and with a small number of samples, MEGA judiciously selects samples that can collect maximum information from users to remove irrelevant k-CNF and k-DNF. MEGA removes irrelevant terms from the query-concept (i.e., a k-CNF), and at the same time, refines the sampling boundary (i.e., a k-DNF) so that the most informative samples can be selected in its subsequent iteration. MEGA uses refinement and sampling in a complementary way to achieve fast convergence to target concepts.

Note that MEGA does not require seeding a query. MEGA presents randomly selected images as the first round of examples. Even if all of the images generated in the first round are irrelevant, MEGA uses the negative images to reduce the set of potentially relevant images substantially (the k-DNF that bounds the sampling space can be shrunk substantially by the negative images). Thus, the probability is high that a relevant image will be generated in the next round. For the

detailed description of MEGA, please refer to [2].

3 Mining Optimal Feature Groupings

The MEGA scheme described so far is not yet concerned with scalability in regard to M (the number of features for depicting an object). In this section, we describe a mining method for tackling the *dimensionality-curse* problem.

The number of disjunctions in a k-CNF (and, likewise, the conjunctives in a k-DNF) can be written as

$$\sum_{i=1}^{k} \binom{M}{i}. \tag{1}$$

When M is large, a moderate k can result in a large number of disjunctive terms in a k-CNF, which causes high space and time complexity for learning. For instance, an image database that we have built [1] characterizes each image with 144 features ($M = 144$). The initial number of disjunctions in a 3-CNF is half a million, and in a 4-CNF is eighteen million.

To reduce the number of terms in a k-CNF, we divide a learning task into G sub-tasks, each of which learns a subset of the features. Dividing a feature space into G subspaces reduces both space and time complexity by a factor of $O(G^{k-1})$. For instance, setting $G = 12$ in our image database reduces both space and time complexity for learning a 3-CNF by 140 times (the number of terms is reduced to $3,576$), and for learning a 4-CNF by $1,850$ times (the number of terms is reduced to $9,516$). The savings is enormous in both space and learning time. (The wall-clock time is less than half a second for one learning iteration for a 4-CNF concept on a Pentium-III processor.)

This divide-and-conquer approach may trade precision for speed, since some terms that involve features from more than one feature subset can no longer be included in a concept. For instance, if we divide features a, b, c, and d into two groups $\{a, b\}$ and $\{c, d\}$, a 2-CNF can no longer have the following four terms: $a \vee c$, $a \vee d$, $b \vee c$, and $b \vee d$. Missing terms may or may not hurt search accuracy. If a query concept does not contain the missing terms, its search accuracy is not affected; conversely, its accuracy decreases. If we know a term is not relevant to any search concept, we can comfortably separate its variables into different groups.

In the remainder of this section, we propose using a genetic-based algorithm to mine the best feature groupings so that the missing terms have a minimum effect on search accuracy.

3.1 The Genetic-based Algorithm for Mining Grouping

We apply genetic algorithms (GAs) [5] to search the best feature grouping. GAs are search algorithms based on an analogy to biological evolution. The simple GAs involve three operations: *selection*, *crossover*, and *mutation*. By applying these operations, GAs repeatedly update the current population. On each iteration, each individual of the population is evaluated according to a *fitness function*, which depends on the application. The objective of the simple GAs is to evolve a population of individuals having high fitness values.

In this section, we present our genetic-based algorithm for searching the best feature groupings. To use GAs, we need to address three issues.

1. Choosing an encoding scheme to translate a hypothesis in the solution space to an individual in GAs, and vice versa.

2. Determining the appropriate genetic operators that meet our objectives.

3. Selecting a good fitness function, since the results of GAs depend on the chosen fitness function.

Suppose there are M image features that we would like to divide into G groups. We further limit the search space by forcing the proposition that a valid grouping solution contains exactly M features and no feature is replicated. An individual is defined as such a collection of M features. Therefore, each individual represents a feature-grouping solution. Here, we do not require features inside groups to be ordered. Given an individual, an offspring can be generated by applying *selection*, *crossover*, and *mutation* operations. We defined these genetic operators as follows:

- *Selection*. A portion of the next population is generated by selection within the current population. Selection is biased toward individuals with higher fitness values. We apply the tournament selection without placement as the selection process [9]. It separates the current population into two groups, a "better" group and a "worse" group. This selection scheme repeatedly picks two individuals from the pool and puts the one with higher fitness into the better group and the other one into the worse group until the pool is empty. The selection process is repeated once more. The new population is obtained after putting together the two better groups. This selection technique guarantees that the individual with the highest fitness value will be selected twice, and the individual with the lowest fitness value will not be selected at all.

- *Crossover.* The crossover operator produces a new offspring from a parent by exchanging features between feature groups of the parent. Given a parent, the crossover operator randomly selects two feature groups of the parent, randomly determines the number of features to be exchanged, randomly selects the features to be exchanged, and then exchanges those features between two selected groups. The other $G-2$ feature groups remain unchanged. After the crossover operation, the new offspring still contains exactly M features, and only two feature groups are updated from parent. This operator allows GAs to explore different feature combinations inside a feature group.
- *Mutation.* The mutation operator perturbs the genes of an individual, with a very small probability. Usually, the mutation operator is applied after the crossover operator. Given an individual, the mutation operator decides if the individual is going to be mutated based on the mutation probability. If yes, the mutation operator randomly chooses a feature from a feature group and moves it to another feature group. This operator allows GAs to explore different grouping sizes.

Since a better feature grouping should give higher search accuracy, we define the fitness function of a feature grouping as its search accuracy. We can estimate a grouping's search accuracy through simulation using training data. For instance, we use an image dataset consisting of $2,300$ images of 18 categories. We treat each category as a query concept. MEGA learns each of these 18 query concepts and returns the top-K images which best match each learned concept. The learned concept may be different from the actual concept. We measure search accuracy as the fraction of the top-K images retrieved based on the learned concept actually belonging to the concept (category). For example, a bear query returns top-20 images that meet the bear concept learned by MEGA. The percentage of these 20 images that actually belong to the bear category represents the precision, and thus the fitness, of the query. We estimate the fitness of a grouping by taking the average precision of the top-K retrieval (using the concepts learned based on that grouping) over 18 categories.

We apply the genetic algorithm to discover the optimal groupings as follows: In the initialization phase, the algorithm randomly creates a population with N individuals (i.e., groupings). In the evolution phase, we first compute the fitness value for each grouping by running extensive simulations on the image dataset. The best grouping is assigned as the grouping with the highest fitness value. In the next step, we apply tournament selection without replacement to generate some offspring groupings. The other offspring groupings are created using crossover operator.

Mutation operation is then applied to a certain fraction (e.g., 0.1 in our experiments) of the offspring. We then update the population with the offspring and repeat the same process until it reaches the desired number of generations. At the end, the genetic algorithm reports the best grouping(s) it found during the evolution process. We present our discovery of the optimal feature grouping in Section 4.

4 Experiments

In this section we report our experimental results. The main goals of our experiments were to

1: Study the tradeoff between learning time and learning accuracy (Sections 4.1 and 4.2).

2: Find ways to group features so that high retrieval precision can be achieved within a few iterations (Section 4.3).

3: Examine which features are important for different query concepts (Section 4.4).

We conducted experiments on a 2,300-image dataset collected from Corel image CDs and the Internet. Each image was characterized by a 144-dimension feature vector, which contained information about color histograms, color moments, textures, etc.

Each of the query concepts we tested belongs to one of the eighteen image categories: *architecture*, *bears*, *big cities*, *clouds*, *elephants*, *fabrics*, *fireworks*, *flowers*, *food*, *landscapes*, *people*, *objectionable images*, *stars*, *textures*, *tigers*, *tools*, and *waves*. Many categories were added to make the learning task challenging. For instance, the *big cities* and *architecture* categories both have buildings. Three animal categories —*bears*, *elephants*, and *tigers*— all have landscape background which makes the discernment of these categories difficult. The *fireworks* and *stars* categories both share the same dark sky background. *Clouds* and *waves* can appear very similar to each other, as can *flowers* and *fabrics*. Finally, various texture images (e.g., skin, brick, grass, water, etc.) were added to raise the learning difficulty for all categories.

We used *precisions* of the top-10 and top-20 retrievals to measure performance. Since in the online image query scenario, 10 and 20 are practical numbers of images to present to a user, and the ratio of these images fitting the user's target concept is of major concern. We tallied precision for up to only five iterations, since we deemed it unrealistic to expect an interactive user to conduct more than five rounds of relevance feedback. To be objective, we ran each experiment ten times.

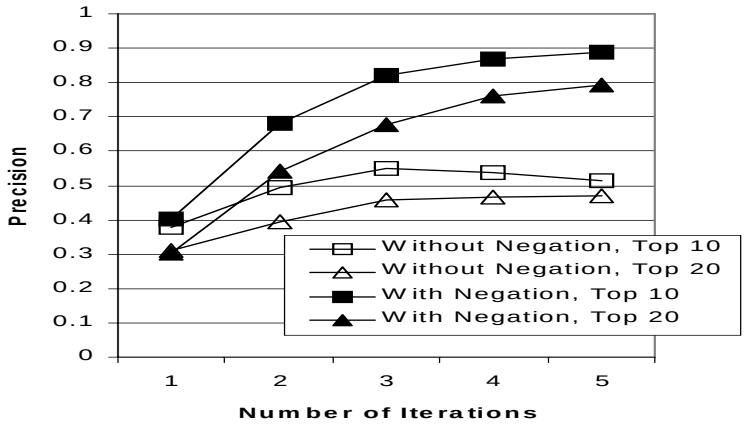


Figure 2: The Effect of Negations.

4.1 The Effect of Negations in Boolean Formulae

As previously stated, each of our images was characterized by a 144-dimension vector. Each feature has a corresponding predicate. We further expanded the method to include negations of these predicates. In other words, the boolean variables in a Boolean formula not only can be of form P_{x_i}'s, but also can be their negations, ($\neg P_{x_i}$'s). We first ran experiments without including negations of predicates and then included them with all other conditions fixed. The average top-10 and top-20 retrieval precisions are compared in Figure 2.

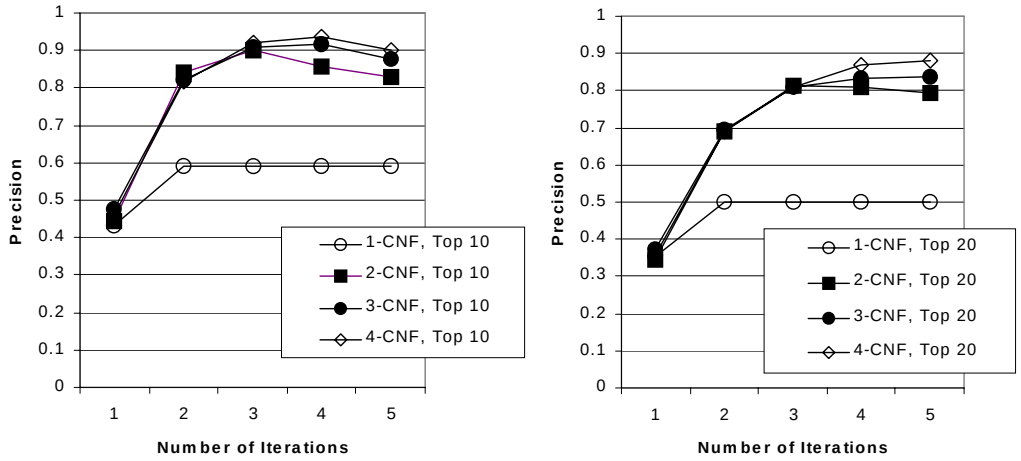
The figure shows that the precisions of both top-10 and top-20 retrievals are significantly increased when negations are included. This is because many query concepts can be expressed only with negations. For instance, an *elephant* image usually does not involve red color. The negation of the predicate "red high" can be useful for distinguishing an elephant image from, say, a flower image that may satisfy the "red high" predicate. Since including negations can substantially improve retrieval precision, we employ negations in our system.

4.2 The Effect of k and G

We model query concepts as k-CNF. The larger the value of k, the more complex concepts we can model. However, learning a more complex concept requires higher space and computational complexity. We should find a k that can adequately capture the query concepts for our applications, and can also be learned in a reasonable amount of space and time.

We experimented with different values of k, from $k = 1$ to $k = 4$. Figure 3 shows the average

precisions of top-10 and top-20 retrievals. Figure 4 shows the running time for employing different k's.



(a) Top-10 Retrieval (b) Top-20 Retrieval

Figure 3: The Effect of k on Precision

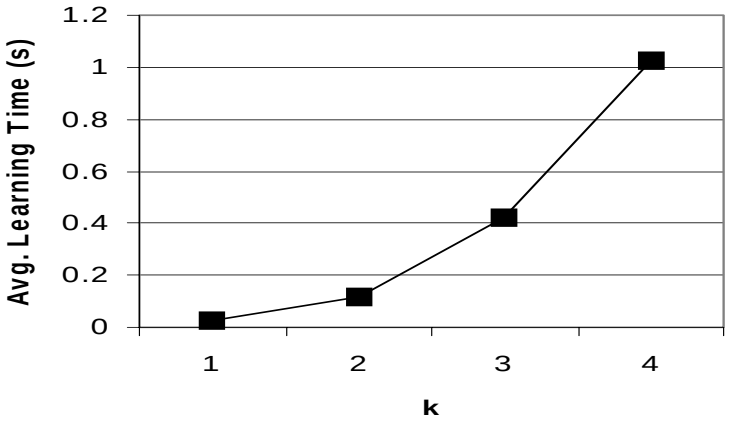


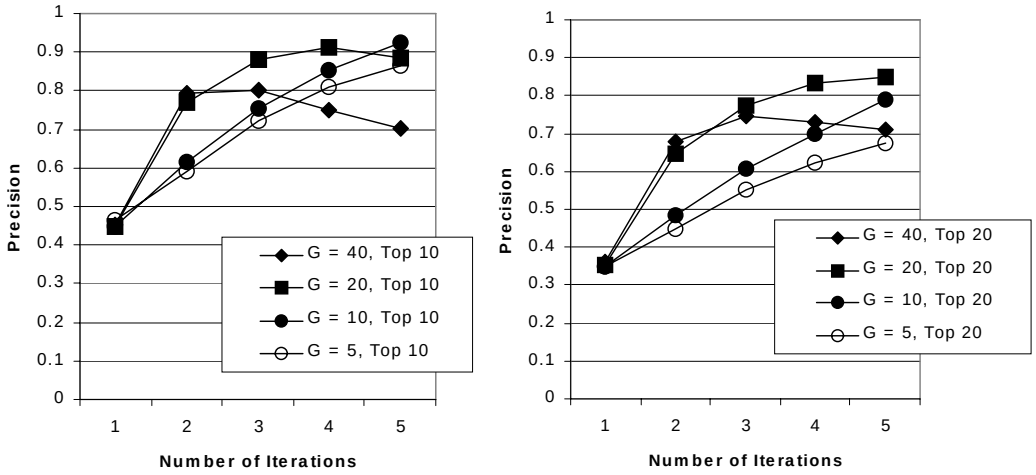
Figure 4: The Effect of k on Learning Time.

Since the trends of the top-10 and top-20 retrievals are similar, we discuss only the top-10 results. For 1-CNF, although the learning time is very short, the achieved search precision is under 60%. After we increased k to two, precision reached 90% after just three iterations. The learning time increased to about 0.1 second, which is satisfactory. When we further increased k to three, the precision surpassed 90% after four iterations, and the learning time (0.4 second) was still low. Further increasing k to four would not be productive, since the precision gain is marginal but the learning time increases substantially. For this image dataset, it is thus sufficient

to model query concepts in 3-CNF.

Next, we examine the effect of G. In section 3 we discussed dividing features into G groups to speed up learning time. We experimented on different G values: 40, 20, 10 and 5.

Figures 5 and 6 show that tradeoffs exist between running time and search accuracy. When G is large ($G = 40$), the average running time is short (0.2 second), but the achieved precision can be dismal (around 80% for top-10 retrieval). At the other extreme, when we set $G = 5$, the precision is good but the running time is unacceptably high. When features are divided into too many groups, we are left with fewer terms to represent some query concepts. When features are divided into too few groups, the number of terms can be very large and hence would require lengthy learning time. Setting G to between 10 and 20 seems to give us a reasonable tradeoff between precision and running time. In the remainder of our experiments, we thus divided features into $G = 20$ groups.



(a) Top-10 Retrieval

(b) Top-20 Retrieval

Figure 5: The Effect of G on Precision.

4.3 Discovering Optimal Groupings

Based on the above experimental results, we set G to 20 and model query concepts in 3-CNF. In this subsection, we are interested in finding how features should be divided into $G = 20$ groups.

We used the genetic algorithm presented in Section 3 to learn the optimal groupings. We

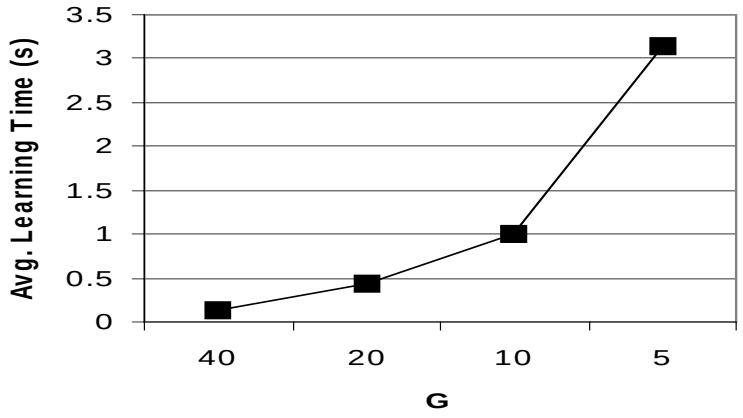


Figure 6: The Effect of G on Learning Time.

started with 40 randomly generated groupings. We ran experiments on each of them. Based on their results (fitness), we generated the next generation of groupings. We ran the experiment up to 120 generations. Figure 7 plots the precision that can be achieved in those 120 generations.

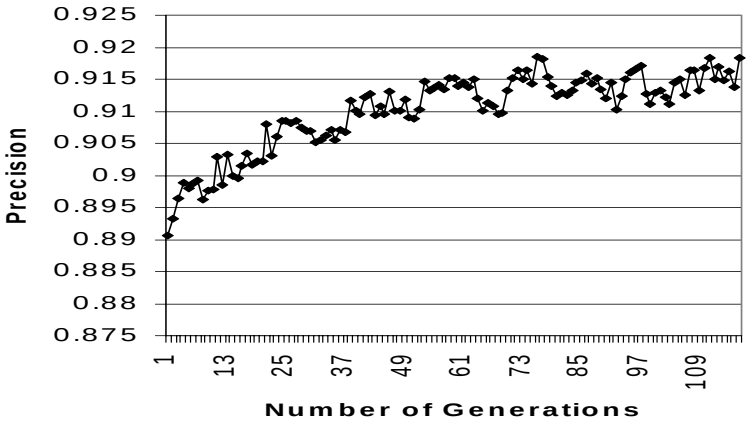


Figure 7: Average of the Highest Precisions for Each Generation.

From Figure 7, we can see that the genetic algorithm increases the precision from 89% to 92%, which is not an insignificant boost. To understand the factors for achieving that improvement, we also studied the average intra-group feature correlation of each generation, which is presented in Figure 8. From both Figures 7 and 8 we can identify a pattern: that having a low intra-group correlation tends to toward a higher search precision. To verify that our observation was valid, we conducted extensive experiments on precision achieved by different intra-group correlations. Figure 9 shows that the low intra-group correlation grouping can achieve a 92% precision, whereas

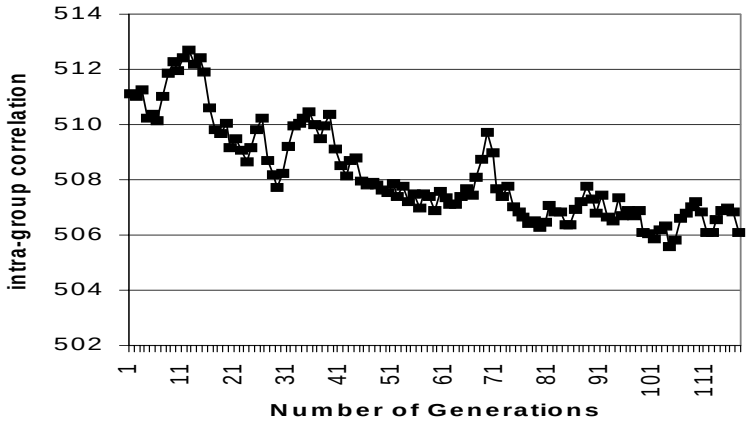


Figure 8: Average of Intra-group correlations for Each Generation.

the high intra-group correlation grouping can achieve only 85% precision.

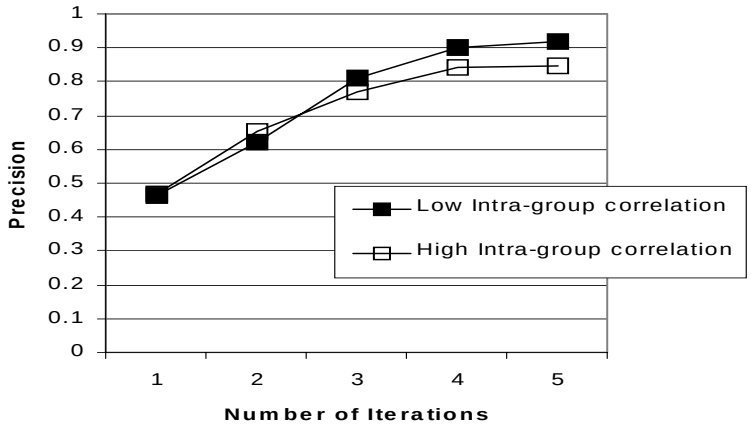


Figure 9: Precision of Top-10 Retrieval for Different Grouping Schemes.

Remarks

We have shown that the loss of precision can be reduced by organizing a feature space intelligent. The terms *feature resolution* and *feature correlation* (a weak form of feature resolution) defined below can help us understand the reasons.

Definition 3: **Feature resolution**: Feature P_i is said to have higher resolution than feature P_j if the presence of P_i implies the presence of P_j (or the absence of P_j implies the absence of P_i). Let $P_i \in P_j$ denote that P_i has higher resolution than P_j. We say that $P_i \in P_j$ if and only if the

conditional probability $P(P_j|P_i) = 1$.

Definition 4: **Feature correlation**: A feature P_i is said to have high correlation with feature P_j if the presence of P_i implies the presence of P_j (and vice versa) with high probability. We say that $P_i \sim P_j$ if and only if the conditional probability $P(P_j|P_i)/P(P_j) = P(P_i|P_j)/P(P_i) \geq \delta$.

We believe that the following three grouping heuristics can achieve fast and accurate learning.

1. Inter-group multi-resolution features. If features can be divided into groups of different resolutions, we need not be concerned with terms that involve inter-group features. This is because any inter-group terms can be subsumed by intra-group terms. Formally, if P_i and P_j belong to two feature groups and $P(P_i|P_j) = 1$, then $P_1 \vee P_2 = P_2$ and $P_1 \wedge P_2 = P_1$.

2. Intra-group multi-resolution features. Within a feature group, the more predicates involved in a disjunctive term, the lower the resolution of the term. Conversely, the greater the number of predicates involve in a conjunctive term, the higher the resolution of that term. For instance, in a 2-CNF that has two predicates P_1 and P_2, term P_1 and term P_2 have a higher resolution than the disjunctive term $P_1 \vee P_2$ and a lower resolution than the conjunctive term $P_1 \wedge P_2$. The presence of P_1 or P_2 makes the presence of $P_1 \vee P_2$ useless. Base on this heuristic, MEGA examines a term only when all terms with higher resolution have been eliminated.

3. Minimizing intra-group feature correlation. After exploring feature resolution, we can explore the weak form of resolution: feature correlation. According to our empirical study, minimizing intra-group feature correlation tends to improve search accuracy.

4.4 Concepts Learned

In previous subsections we explored how to enhance the retrieval performance by including negations of predicates, selecting appropriate values for k and G, and carefully dividing features into groups. In this subsection, we examine the query concepts we learned for several categories. A learned concept was in 3-CNF, which is a conjunction of disjunctions of at most three predicates. We categorized predicates into three kinds:

1. *First order predicate*. A predicate that appears by itself as a term.

2. *Second order predicate*. A predicate that is not a first order predicate but appears in at least one disjunctive term where the predicate is one of the two literals.

3. *Third order predicate.* A predicate that is neither a first order predicate nor a second order predicate but appears in at least one disjunctive term where the predicate is one of the three literals.

For instance, in the following concept:

$$P_1 \wedge (P_2 \vee P_3) \wedge (P_4 \vee P_5 \vee P_6) \wedge (P_2 \vee P_4 \vee P_7),$$

P_1 is a first order predicate, P_2 and P_3 are second order predicates, and P_4, P_5, P_6 and P_7 are third order predicates. An object has to satisfy P_1 to satisfy the concept. An object does not need to satisfy P_2 to satisfy the concept. It just needs to satisfy one of P_2 and P_3. Therefore, we can say that a first order predicate plays a more important role in a concept than a second order predicate, and a second order predicate plays a more important role in a concept than a third order predicate.

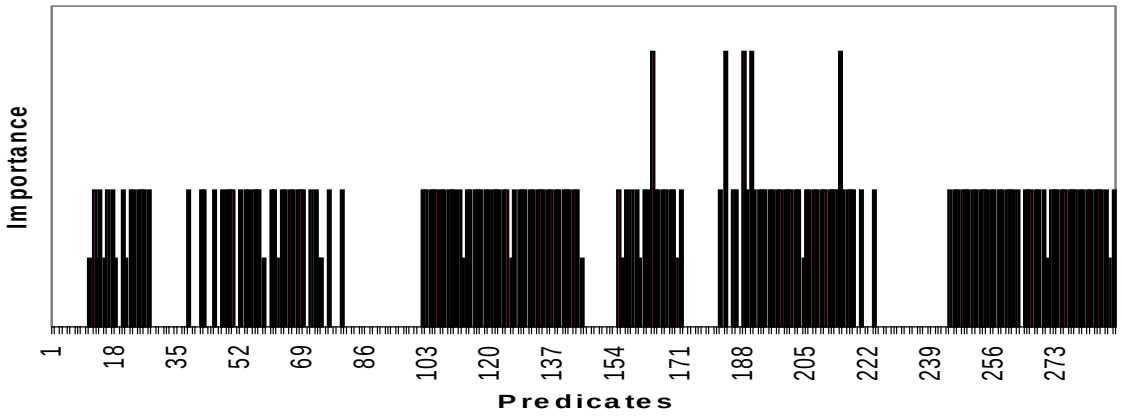


Figure 10: The Architecture Concept.

We present the results of five representative concepts that we learned: *architectures*, *bears*, *clouds*, *elephants*, and *fabrics*. Figure 10 displays the concept learned for *architectures*. The x-axis in the figure depicts the predicate number. Predicates 1 through 144 are the 144 features we use to characterize images. (These 144 features are described in Table 1.) Predicates 145 through 288 are negations of the first 144 predicates. The y-axis in the figure indicates the *relative importance* of a predicate in the concept. A feature plot like Figure 10 can provide us a rough idea about what features represent a concept. Figure 11 shows the feature plots of *bears*, *clouds*, *elephants*, and *fabrics*. Comparing the figures of the architecture concept and the cloud concept, we can quickly tell that the architecture concept has various colors and is texture rich. On the other hand, the cloud concept is represented by only a couple of colors and by many negation texture

predicates.

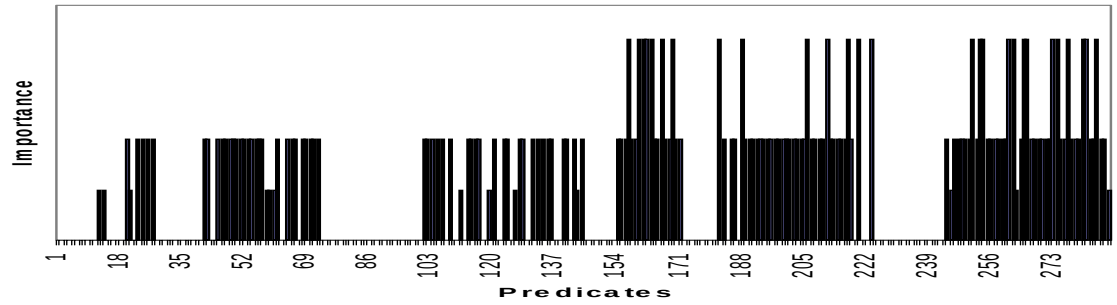
	Color Features (features 1 to 108)	*Texture Features (features 109 to 144)*
Channels	Twelve color channels: black, gray, white, red, yellow, green, blue, brown, purple ,pink, orange and outlier color.	Nine texture channels: vertical coarse texture, horizontal coarse texture, diagonal coarse texture, vertical medium texture, horizontal medium texture, diagonal medium texture, vertical fine texture, horizontal fine texture and diagonal fine texture.
Detail Information	Each color channel has nine features: histograms, spreadness, elongation, & color averages, saturation and intensity in hue, saturation and intensity.	Each texture channel has four features: total energy, energy variance, texture spreadness and texture elongation.

Table 1: Feature Description.

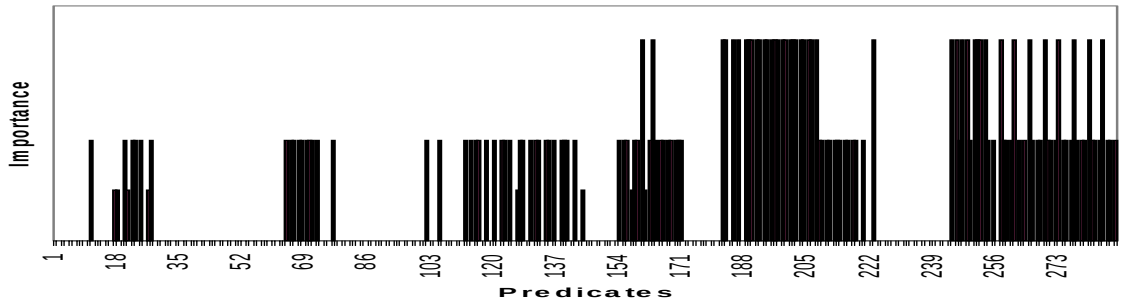
	Channels	*Architecture*	*Bears*	*Clouds*	*Elephants*	*Fabrics*
Colors	Red color channel	–	–	–	–	+
	Yellow color channel	–	+	–	+	–
	Blue color channel	+	–	+	–	–
	Brown color channel	–	–	–	+	–
Textures	Vertical coarse texture		–	–	–	+
	Horizontal coarse texture	+	+	–	+	+
	Diagonal coarse texture	+	–	–	+	+
	Vertical medium texture	+	–	–	+	+
	Horizontal medium texture	+	–	–	+	–
	Vertical fine texture	+	–	–	+	+
	Horizontal fine texture	+	–	–	–	–
	Diagonal fine texture	+	–	–	–	+

Table 2: Summary of the Five Categories.

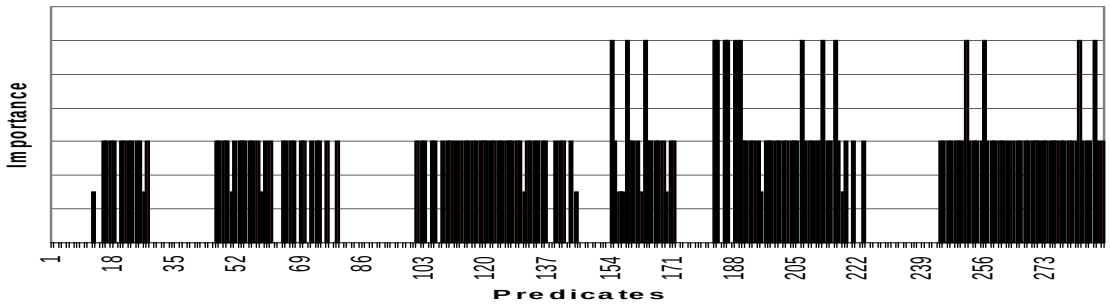
Table 2 highlights key feature-concept associations of the five concepts (categories). A plus sign in a cell indicates the feature on that row and the concept on that column has a strong association. A negative sign indicates that the feature on that row and the concept on that column has a strong negative association. For example, in the color channels, the blue color has a strong association with architectures and clouds but has a strong negative association with the rest of the concepts. Since most architecture and all cloud images in our dataset have blue sky on the background, this association makes sense. In the texture channels, almost all features are strongly associated with architectures, but are negatively associated with cloud images. It is interesting to examine the feature-concept associations of bears and elephants, since they share the similar landscape backgrounds. In the color channels, these two categories are similar except in the brown channel. Brown is strongly associated with elephants but is not with bears. The major difference between elephants and bears comes from the textures: The elephant category is associated with various textures, whereas the bear category is only strongly associated with the



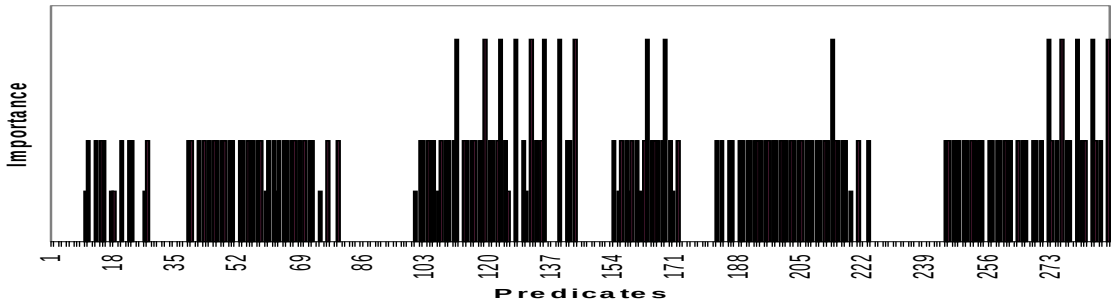
(a) The Bears Concept



(b) The Clouds Concept



(c) The Elephants Concept



(d) The Fabrics Concept

Figure 11: Concepts of the Four Categories.

horizontal coarse texture channel.

4.5 Observations

We make the following observations from our image-concept mining results:

1. *Including negations.* It is critical to include both predicates and their negations in a boolean formula. Including negations can express complex concepts more accurately and hence enhance search accuracy.

2. *Setting k to achieve good search accuracy and time tradeoff.* For our dataset, it is sufficient to model query concepts in 3-CNF to achieve high precision at low search cost. In general, we need to choose a proper k through empirical study in order to adequately model query concepts while keeping the computational cost low.

3. *Choosing G to achieve good search accuracy and space/time complexity tradeoff.* For our dataset, it is desirable to divide features into 20 groups. We need to choose a proper G to speed up learning time while maintaining high search accuracy.

4. Organizing features to minimize intra-group correlation for maintaining high search accuracy. The genetic algorithm is effective in finding feature groupings that yield better search accuracy. The groupings with smaller intra-group correlation tend to yield higher search accuracy.

5. Understanding feature-concept associations for measuring similarity. From concept mining, we can identify what features are important for an image concept. This information can be used for measuring image similarity. For instance, if we know that the search concept is clouds, we can weight the blue channel and negation texture predicates heavier to achieve better search results.

5 Conclusion

In this paper, we have presented two mining problems: learning users' query concepts and discovering optimal feature groupings. Our mining results showed that when parameters k and G are properly selected, our learning algorithm (MEGA) can fuzzily capture a k-CNF query concept

usually in two to three feedback iterations and can comprehend a target concept very well in three to five iterations.

To speed up learning and to reduce the learning space requirement, we proposed a divide-and-conquer method that divides the concept-learning task into G subtasks. We noticed that a task must be divided carefully, or search accuracy may suffer. We used a genetic-based mining algorithm to discover good feature groupings. Through analysis and mining results, we observed that organizing image features in a multi-resolution manner, and maximizing inter-group feature correlation (or minimizing intra-group correlation), can speed up query-concept learning by $O(G^{k-1})$ folds, while maintaining high search accuracy. We also presented the discovered k-CNF expressions for various image categories. We believe that the concept-feature associations learned can be used for enhancing image search accuracy.

References

[1] E. Chang and T. Cheng. Perception-based image retrieval. *ACM Sigmod (Demo)*, May 2001.

[2] E. Chang and B. Li. Mega — the maximizing expected generalization algorithm for learning complex query concepts (extended version). *Technical Report http://www-db.stanford.edu/∼echang/mega.ps*, November 2000.

[3] C. Chen. *Fuzzy Logic and Neural Network Handbook*. McGraw-Hill, 1996.

[4] R. Fagin. Fuzzy queries in multimedia database systems. *ACM Sigacr-Sigmod-Sigart Symposium on Principles of Database Systems*, 1998.

[5] D. E. Goldberg and R. Burch. *Genetic Algorithms in Search, Optimization, and Machine Learning, Reading*. Addison-Wesley, 1989.

[6] M. Kearns, M. Li, and L. Valiant. Learning boolean formulae. *Journal of ACM*, 41(6):1298–1328, 1994.

[7] M. Kearns and U. Vazirani. *An Introduction to Computational Learning Theory*. MIT Press, 1994.

[8] T. Michell. *Machine Learning*. McGraw Hill, 1997.

[9] B. L. Miller and D. E. Goldberg. Genetic algorithms, tournament selection, and the effects of noise. *Complex Systems*, pages 193–212, 1995.

[10] Y. Rui, T. S. Huang, and S.-F. Chang. Image retrieval: Current techniques, promising directions and open issues. *Journal of Visual Communication and Image Representation*, March 1999.

[11] L. A. Zadeh. Fuzzy sets. *Information and Control*, pages 338–353, 1965.